

E-commerce Backend system

Objective: The primary objective of this project is to secure sensitive user data and it ensures data retrieval. So that administration passwords can be stored and it verify the shipping records during delivery process

Introduction: An E-commerce Backend system using hashing is to manage and process large volume of data such as transactions, user accounts. In e-commerce platforms, scalability is critical, as thousands of users may search products or place orders. To handle this hashing it allows to store data like product ID's, order details can be indexed. This significantly improves the operations such as validating user logins and shopping carts.

Technical stack (technologies used) :

Data structure : Hash table, collision handling techniques

Programming language : java, python

Backend: spring boot(java)

Tool: VS Code (IDE), Mithun.C language

Methodology : Here first user registration and login , product search and filtering are analyzed. Hash functions used to convert keys into indices. Developed using separate chaining using linked list and apply hashing algorithms (like bcrypt). Based on the requirements system is designed by backend architecture using product module. Then program is tested and and perform unit testing for individual modules. Deploy the backend on platforms.

Architecture: It includes web browser or mobile apps send requests such as login, product search, place order. API layer receives the request. API acts as single point , routing to services . This layer follows module like user module, cart module, order module. cache layer is used in memory-storage. It's distribution using hashing , balancing across servers. Seperate data bases (MongoDB) for services like catalog , orders. Here it improves response time.

Conclusion: The E-commerce backend using hashing demonstrate how efficient data structure improve the performance of the modern applications. Successfully address the critical security , integrity. The system protects user credentials against unauthorized access and reduces server load , supports scalability.

Future Enhancement:

1. Distributed Hashing (scalability)
2. Improved collision resolution Techniques
3. Security Enhancement

Submitted by: V.chathurya (25p61a05ah)